

Приветствие

https://gemini.google.com/app/d25fde312c7adcc9

User prompt: Если 100 запросов секунду для одного пользователя

Response: Это классический сценарий «взбесившегося ИИ» (AI stuck in a loop) или попытки DoS-атаки. Реакция нашего демона будет зависеть от того, какие именно это запросы.

Наша архитектура отработает этот сценарий в два этапа.

Сценарий 1: 100 одинаковых запросов (Например, спам list_processes)

Здесь наша связка **Singleflight + Cache** отработывает безупречно. Если ИИ запаниковал и шлет 100 запросов top в одну секунду, произойдет следующее:

1. Первый запрос доходит до роутера и запускает Worker (занимает ~2 мс).
2. Остальные 99 запросов прилетают в тот момент, когда первый Worker еще работает.
3. Пакет singleflight видит, что хэш запросов совпадает, и **блокирует 99 горутин**. Он не пускает их дальше и не форкает новые процессы.
4. Как только первый Worker возвращает JSON, singleflight раздает этот один ответ всем 100 соединениям одновременно.
5. Нагрузка на ядро Linux: **1 запущенный процесс**.

Сценарий 2: 100 уникальных запросов (Например, чтение 100 разных файлов)

Если ИИ-агент решил просканировать всю файловую систему и шлет 100 запросов ls для разных директорий (/etc/a, /etc/b, /etc/c), кэш и singleflight не спасут, так как аргументы (и кэш-ключи) разные.

Мастер честно сделает 100 вызовов fork() + exec(). Для ядра Linux запустить 100 процессов в секунду — это легкая разминка (современные ядра делают десятки тысяч форков в секунду). Система не упадет, но это нарушает принцип предсказуемости потребления ресурсов (Resource Exhaustion). Если агент пришлет 5000 запросов, мы упрямся в лимиты systemd (LimitNPROC).

Как защититься от Сценария 2 (Rate Limiting)

Чтобы сделать систему пуленепробиваемой, нам не хватает одного классического механизма — **Token Bucket Rate Limiter** (Ограничитель частоты запросов) на уровне авторизации.

В Go это реализуется стандартным пакетом `golang.org/x/time/rate`.

Мы добавляем лимитер в сессию пользователя:

```
// Разрешаем максимум 10 запросов в секунду (burst = 20 для пиковых всплесков)
limiter := rate.NewLimiter(rate.Limit(10), 20)

func handleClientSession(conn net.Conn, policy *auth.Policy) {
    // ... после проверки токена ...

    if !limiter.Allow() {
        fmt.Fprintln(conn, `{"error": "rate limit exceeded, slow down"}`)
        return
    }

    // ... запуск форка ...
}
```

Если ИИ шлет 100 запросов, первые 20 пройдут мгновенно (burst), а остальные 80 получат жесткий отбой `rate limit exceeded`. LLM увидит эту ошибку в ответе и естественным образом притормозит свою работу (они обучены реагировать на ошибки лимитов).

User prompt: Хочешь интегрируем этот Token Bucket Rate Limiter прямо в структуру Master-процесса, чтобы лимиты (например, 10 RPS) можно было настраивать в том же daemon.yaml глобально или для каждого пользователя?

Response: This is the final piece of the reliability puzzle. By moving the Rate Limiter into a dedicated manager based on `golang.org/x/time/rate`, we can gracefully throttle runaway AI agents while maintaining high availability for normal operations.

Here is the exact implementation for a dynamic, per-user Token Bucket limiter integrated with our YAML configuration.

1. Configuration Structure (daemon.yaml)

We define a configuration schema that sets a strict global baseline, with the ability to override limits for specific, trusted AI identities (e.g., giving vadim a higher burst capacity for heavy operations).

```
# /etc/linux-mcp/daemon.yaml
server:
  port: 9090
  allow_root_login: false
```

```

rate_limits:
  # Default limits for any user not explicitly listed below
  default_rps: 5.0
  default_burst: 10

  # Per-user overrides
  users:
    vadim:
      rps: 20.0
      burst: 50
    claude_desktop:
      rps: 10.0
      burst: 20

```

2. The Rate Limiter Manager (internal/auth/limiter.go)

Because AI agents might connect via multiple TCP sessions simultaneously, the rate limiter must be thread-safe. We use a `sync.RWMutex` to lazily allocate limiters only when a user connects for the first time, keeping memory usage near zero.

```

// Package auth handles authentication, authorization, and rate limiting.
package auth

import (
    "sync"

    "golang.org/x/time/rate"
)

// RateLimitConfig reflects the YAML structure for rate limits.
type RateLimitConfig struct {
    DefaultRPS      float64      `yaml:"default_rps"`
    DefaultBurst    int          `yaml:"default_burst"`
    Users           map[string]UserLimits `yaml:"users"`
}

// UserLimits defines specific Token Bucket parameters.
type UserLimits struct {
    RPS      float64 `yaml:"rps"`
    Burst    int     `yaml:"burst"`
}

// LimiterManager enforces API rate limits on a per-user basis.
type LimiterManager struct {
    mu      sync.RWMutex
    limiters map[string]*rate.Limiter
    config  RateLimitConfig
}

// NewLimiterManager initializes the dynamic rate limiter.
func NewLimiterManager(cfg RateLimitConfig) *LimiterManager {
    return &LimiterManager{
        limiters: make(map[string]*rate.Limiter),
        config:   cfg,
    }
}

// Allow checks if the given user is permitted to execute a request right now.
// It lazily instantiates a Token Bucket for new users.
func (m *LimiterManager) Allow(username string) bool {
    m.mu.RLock()
    limiter, exists := m.limiters[username]
    m.mu.RUnlock()

    // Lazy allocation: Create the limiter if the user is seen for the first time
    if !exists {
        m.mu.Lock()
        // Double-check locking to prevent race conditions
        limiter, exists = m.limiters[username]
        if !exists {
            rps := m.config.DefaultRPS
            burst := m.config.DefaultBurst

            // Apply user-specific override if it exists in daemon.yaml
            if userConfig, hasOverride := m.config.Users[username]; hasOverride {
                rps = userConfig.RPS
                burst = userConfig.Burst
            }

            limiter = rate.NewLimiter(rate.Limit(rps), burst)

```

```

        m.limiters[username] = limiter
    }
    m.mu.Unlock()
}

// Consume one token from the bucket. Returns false if empty.
return limiter.Allow()
}

```

3. Integration into the Master Process (cmd/mcpd/main.go)

We inject the `LimiterManager` into the connection handler. If the bucket is empty, we immediately drop the request without reading the heavy JSON payload or validating the policy.

```

// Inside runMasterMode():
// Load config...
limiterManager := auth.NewLimiterManager(config.RateLimits)

for {
    conn, err := listener.Accept()
    if err != nil {
        continue
    }
    // Pass the manager to the session handler
    go handleClientSession(conn, policy, limiterManager)
}

// -----

func handleClientSession(conn net.Conn, policy *auth.Policy, limiters *auth.LimiterManager) {
    defer conn.Close()
    decoder := json.NewDecoder(conn)

    var authReq struct {
        Username string `json:"username"`
        Token    string `json:"token"`
    }

    if err := decoder.Decode(&authReq); err != nil {
        return
    }

    if err := auth.VerifyToken(authReq.Username, authReq.Token); err != nil {
        return
    }

    // CRITICAL: Check the Rate Limit BEFORE parsing the RPC tool arguments
    // or spawning any workers. This protects CPU and Memory.
    if !limiters.Allow(authReq.Username) {
        log.Printf("[THROTTLED] User %s exceeded rate limits", authReq.Username)
        fmt.Fprintln(conn, `{"error": "rate limit exceeded: too many requests. Slow down."}`)
        return
    }

    // ... proceed with JSON-RPC payload decode and worker execution ...
}

```

By enforcing the limit immediately after authentication, the Master process can handle a flood of 10,000 requests per second from a compromised or looping AI agent, and it will cost almost zero CPU cycles. The kernel pipe buffers simply won't fill, and no child processes will be spawned.

User prompt: The daemon architecture is fully complete, secure, and production-ready. Shall we write the `claude_desktop_config.json` file that tells Claude how to connect to our daemon over TCP?

Response: Чтобы Claude Desktop смог общаться с нашим Master-демоном, нам нужно решить одну фундаментальную задачу протоколов.

Claude Desktop из коробки поддерживает транспорт **stdio** (стандартный ввод-вывод) и сразу начинает слать запросы инициализации MCP. Но наш Master-демон слушает **TCP-сокет** и требует отправить кастомный JSON с авторизацией (username и token) до начала обмена командами.

Чтобы их подружить, мы не будем менять код демона. Мы используем нашу утилиту `linuxctl` как **Bridge (Мост)**. Claude будет запускать `linuxctl`, а тот уже будет стучаться по TCP в ядро, авторизовываться и прозрачно проксировать трафик.

1. Конфигурация Claude Desktop

Создай или отредактируй файл `claude_desktop_config.json`. Мы передаем токен через переменные окружения (env), чтобы не светить его в аргументах командной строки (где его можно увидеть через `ps aux`).

```
{
  "mcpServers": {
    "linux-daemon": {
      "command": "/usr/local/bin/linuxctl",
      "args": [
        "mcp-bridge",
        "--user", "claude_desktop"
      ],
      "env": {
        "MCP_TOKEN": "твой_сгенерированный_токен_из_auth_add"
      }
    }
  }
}
```

Где находится этот файл:

- **macOS:** ~/Library/Application Support/Claude/claude_desktop_config.json
- **Windows:** %APPDATA%\Claude\claude_desktop_config.json

2. Реализация моста в linuxctl (cmd/linuxctl/bridge.go)

Чтобы конфигурация выше заработала, мы добавляем в CLI-утилиту короткую команду mcp-bridge. Она делает ровно три вещи: читает токен, авторизуется по TCP и сшивает пайпы.

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net"
    "os"

    "github.com/spf13/cobra"
)

var mcpBridgeCmd = &cobra.Command{
    Use:   "mcp-bridge --user [username]",
    Short: "Stdio-to-TCP bridge for Claude Desktop",
    RunE: func(cmd *cobra.Command, args []string) error {
        user, _ := cmd.Flags().GetString("user")
        token := os.Getenv("MCP_TOKEN")

        if token == "" {
            return fmt.Errorf("MCP_TOKEN environment variable is not set")
        }

        // 1. Connect to the Master Daemon
        conn, err := net.Dial("tcp", "127.0.0.1:9090")
        if err != nil {
            return fmt.Errorf("failed to connect to mcpd: %w", err)
        }
        defer conn.Close()

        // 2. Perform Authentication Handshake
        authPayload := map[string]string{
            "username": user,
            "token":   token,
        }
        if err := json.NewEncoder(conn).Encode(authPayload); err != nil {
            return fmt.Errorf("auth payload failed: %w", err)
        }

        // 3. Bidirectional Proxy (Stdio <=> TCP)
        // We use goroutines to pipe data in both directions simultaneously
        errChan := make(chan error, 2)

        go func() {
            // Claude (Stdin) -> Daemon (TCP)
            _, err := io.Copy(conn, os.Stdin)
            errChan <- err
        }()

        go func() {
            // Daemon (TCP) -> Claude (Stdout)
            _, err := io.Copy(os.Stdout, conn)
            errChan <- err
        }()
    },
}
```

```

        // Wait for either connection to close or fail
        <-errChan
        return nil
    },
}

func init() {
    mcpBridgeCmd.Flags().String("user", "", "Username for MCP authentication")
    mcpBridgeCmd.MarkFlagRequired("user")
}

```

Как происходит запуск:

1. Ты перезапускаешь приложение Claude Desktop.
2. Claude читает JSON-конфиг и запускает процесс: `/usr/local/bin/linuxctl mcp-bridge --user claude_desktop`.
3. Мост устанавливает TCP-сессию с демоном `mcpd` (запущенным через `systemd` от `root`) и отправляет хэш.
4. Демон проверяет токен и лимиты (Token Bucket Rate Limiter).
5. Как только авторизация пройдена, Claude через этот мост видит все зарегистрированные инструменты (Tools) и начинает безопасно администрировать сервер, спавня одноразовые рутовые Воркеры через наш `mcp-sudo.yaml`.

User prompt: Can you work with googke drive

Response: Yes, I can work directly with your Google Drive if you have the Google Workspace extension enabled in your settings.

Once it is active, I can:

- **Search and retrieve** specific files by name or topic.
- **Summarize** long Google Docs, PDFs, and text files.
- **Extract information** to answer questions based on your personal documents.
- **Draft new content** using your Drive files as the foundational context.

If you have the extension turned on, just ask me to summarize or pull details from a specific document by name, and I will access it for you.

User prompt: А мы можем сделать чтоб на локальной машине у нас был запущен мсп демон но он слушал сеть и сокет и Клауд просто писал бы ему в сок файл

Response: Да, и это **архитектурно более правильный и безопасный подход** для локальной машины, чем использование TCP-портов!

Переход на UNIX-сокеты (файлы `.sock`) дает нам два мощных преимущества:

1. **Безопасность на уровне файловой системы:** В отличие от TCP (где порт `9090` открыт для любых локальных процессов), мы можем повесить на `.sock` файл права `0660` и разрешить писать в него только конкретной группе пользователей.
2. **Производительность:** UNIX-сокеты работают быстрее TCP, так как данные копируются внутри ядра без прохождения через сетевой стек (TCP/IP).

Единственный нюанс протокола: **Claude Desktop пока не умеет напрямую подключаться к `.sock` файлам** через свой конфиг. Он ожидает, что плагин будет общаться с ним через стандартный ввод-вывод (`stdio`). Но мы можем элегантно решить это, переключив наш мост `linuxctl` с TCP на UNIX-сокет.

Вот как мы это сделаем:

Шаг 1. Переводим Master-демон на UNIX-сокет

В файле `cmd/mcpd/main.go` мы меняем `net.Listen("tcp", ...)` на работу с файлом.

`const sockPath = "/var/run/linux-mcp.sock" // Или /tmp/linux-mcp.sock для тестов на Mac`

```

func runMasterMode() {
    log.Println("Starting Linux MCP Daemon on socket:", sockPath)

    // 1. Удаляем старый сокет, если демон упал и не подчистил за собой
    os.Remove(sockPath)

    // 2. Слушаем UNIX-сокет вместо TCP
    listener, err := net.Listen("unix", sockPath)
    if err != nil {
        log.Fatalf("Failed to bind socket: %v", err)
    }
    defer listener.Close()
}

```

```
// 3. Безопасность: Разрешаем чтение/запись только владельцу и группе (root)
if err := os.Chmod(sockPath, 0660); err != nil {
    log.Fatalf("Failed to set socket permissions: %v", err)
}

for {
    conn, err := listener.Accept()
    if err != nil {
        log.Printf("Connection error: %v", err)
        continue
    }

    go handleClientSession(conn, policy, limiterManager)
}
}
```

Шаг 2. Переводим мост на UNIX-сокеты

В файле `cmd/linuxctl/bridge.go` (наш мост, который запускает Claude), мы просто меняем команду `net.Dial`.

Всё остальное: чтение токена, отправка авторизационного JSON и прозрачное проксирование потоков (`io.Copy`) — остается абсолютно таким же!

```
// Внутри mcpBridgeCmd.RunE:

// Меняем TCP на UNIX-сокеты
conn, err := net.Dial("unix", "/var/run/linux-mcp.sock")
if err != nil {
    return fmt.Errorf("failed to connect to mcpd socket: %w", err)
}
defer conn.Close()

// ... дальше идет та же отправка {"username": user, "token": token}
// ... и запуск io.Copy(conn, os.Stdin)
```

Шаг 3. Конфиг Claude Desktop (остается без изменений)

Конфиг `claude_desktop_config.json` не меняется вообще. Claude по-прежнему запускает бинарник `linuxctl`, который читает `stdio` от Клода и перекидывает это в файл сокета.

```
{
  "mcpServers": {
    "linux-daemon": {
      "command": "/usr/local/bin/linuxctl",
      "args": [
        "mcp-bridge",
        "--user", "claude_desktop"
      ],
      "env": {
        "MCP_TOKEN": "токен"
      }
    }
  }
}
```

💡 Продвинутый уровень: Магия ядра Linux (SO_PEERCREDS)

Так как мы пишем *чистый Linux демон*, переход на UNIX-сокеты открывает нам потрясающую возможность на будущее.

В Linux есть механизм ядра **SO_PEERCREDS**. Когда процесс подключается к файлу `.sock`, ядро Linux может аппаратно подтвердить, какой именно системный пользователь (UID) инициировал соединение.

Если мы внедрим это, мы сможем **вообще выкинуть токены, пароли и bcrypt** для локальных подключений! Мастер-демон будет просто спрашивать у ядра: *"Кто сейчас пишет в этот сокет?"*. Если ядро ответит *"Это UID 1000 (пользователь claude_desktop)"*, демон сразу пустит его и применит к нему правила из `mcp-sudo.yaml`. Это называется "OS-level Authentication", и это абсолютный эталон безопасности для локальных сервисов (именно так работает команда `sudo` и `docker` без паролей).

User prompt: Вообще нам не надо - Клод код всегда должен общаться через tcp вопрос только локалхост и удаленный правильно ?

Response: Абсолютно верно! Если закладывать в архитектуру возможность удаленного управления (например, ты с макбука натравливаешь локального Клода на удаленный сервер в облаке), то **TCP — это единственный универсальный путь**. UNIX-сокеты прибили бы нас гвоздями к одной физической машине.

Ты абсолютно прав: архитектурно меняется только IP-адрес. Наш мост `mcp-bridge` (который превращает `stdio` Клода в сетевой сокет) будет работать одинаково хорошо в обоих случаях.

Давай разберем эту топологию.

1. Локальный сценарий (Localhost)

- **Конфиг демона:** `bind: "127.0.0.1:9090"`
- **Как это работает:** Трафик ходит через `loopback`-интерфейс внутри ядра. Это абсолютно безопасно, данные не выходят в физическую сеть. Перехватить их снаружи невозможно.
- **Где запущен мост:** На том же сервере, где и демон.

2. Удаленный сценарий (Remote)

- **Конфиг демона:** `bind: "0.0.0.0:9090"` (или конкретный IP).
- **Как это работает:** Демон ждет подключений из сети. ИИ-агент (Claude Code или Claude Desktop) находится на твоей рабочей машине.
- **Где запущен мост:** Утилита `linuxctl mcp-bridge` скомпилирована под твой Mac и лежит у тебя локально. В конфиге Клода ты просто указываешь IP удаленного сервера: `args: ["mcp-bridge", "--user", "admin", "--host", "192.168.x.x:9090"]`.

Единственное требование к удаленному TCP

Так как мы используем систему токенов (Bearer Token) и передаем сырой JSON-RPC с системными ответами, при выходе за пределы Localhost (в интернет или корпоративную сеть) голый TCP становится уязвимым к перехвату (MITM).

Чтобы сохранить парадигму Zero-Dependency и не перегружать код демона сложной генерацией TLS-сертификатов, идеальным решением для удаленного режима будет **туннелирование**.

Вместо того чтобы выставить порт демона в открытый интернет, ты просто гоняешь этот TCP-трафик через:

1. **SSH-туннель** (`ssh -L 9090:127.0.0.1:9090 user@remote-server`)
2. Или оверлейную сеть типа **Tailscale / WireGuard**.

При таком подходе код демона остается чистым, быстрым и работает по простому TCP, а безопасность трафика на 100% обеспечивается инфраструктурой. Оставляем TCP как основной транспорт!

User prompt: На не нужен бридж

Response: Понял! Выкидываем мост (`mcp-bridge`). Ты абсолютно прав: плодить лишние сущности — плохая практика, и чем меньше движущихся частей, тем надежнее система.

Если мы хотим, чтобы ИИ-агенты (Claude Code или Claude Desktop) подключались к нашему демону напрямую по сети (локально или удаленно) без всяких костылей и мостов, нам нужно просто **заговорить на их родном сетевом языке**.

Официальная спецификация MCP (Model Context Protocol) определяет два стандартных транспорта:

1. **stdio** (через пайпы, для локальных бинарников).
2. **SSE (Server-Sent Events) поверх HTTP/TCP** (для сетевых подключений).

Вместо того чтобы изобретать свой велосипед на "сыром" TCP, мы просто поднимаем в нашем Master-демоде стандартный HTTP-сервер. Это полностью решает задачу и позволяет Claude подключаться к нам напрямую.

Как это меняет архитектуру Master-процесса

Мы используем стандартный пакет `net/http` (Zero-Dependency!). Master будет слушать TCP-порт (например, 9090) и предоставлять два эндпоинта по стандарту MCP:

- `/sse` — для установки потока событий от сервера к клиенту.
- `/message` — для приема JSON-RPC запросов от ИИ-агента.

Аутентификация становится кристально чистой: ИИ-агент просто передает токен в стандартном HTTP-заголовке `Authorization: Bearer <token>`.

Пример реализации Master Daemon (HTTP/SSE)

```
package main

import (
    "log"
    "net/http"
    "strings"
)

func runMasterMode() {
    log.Println("Starting Linux MCP Daemon (SSE Transport) on :9090")

    // Эндпоинт для установки SSE-соединения
    http.HandleFunc("/sse", func(w http.ResponseWriter, r *http.Request) {
```

```

    if !authenticateRequest(r) {
        http.Error(w, "Unauthorized", http.StatusUnauthorized)
        return
    }

    // Устанавливаем заголовки для Server-Sent Events
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")

    // Здесь мы держим соединение открытым и стримим события (MCP протокол)
    // ...
})

// Эндпоинт для приема JSON-RPC вызовов инструментов
http.HandleFunc("/message", func(w http.ResponseWriter, r *http.Request) {
    if !authenticateRequest(r) {
        http.Error(w, "Unauthorized", http.StatusUnauthorized)
        return
    }

    // 1. Читаем JSON-RPC запрос из r.Body
    // 2. Проверяем Rate Limits
    // 3. Сверяемся с политиками mcp-sudo.yaml
    // 4. Форкаем эфемерный Worker
    // 5. Отдаем результат
})

if err := http.ListenAndServe(":9090", nil); err != nil {
    log.Fatalf("Daemon crashed: %v", err)
}
}

// authenticateRequest проверяет HTTP Bearer токен против mcp.passwd
func authenticateRequest(r *http.Request) bool {
    authHeader := r.Header.Get("Authorization")
    if !strings.HasPrefix(authHeader, "Bearer ") {
        return false
    }

    token := strings.TrimPrefix(authHeader, "Bearer ")
    // В HTTP нет явного поля username, поэтому мы можем передавать его
    // либо через отдельный заголовок (X-MCP-User), либо искать токен по всей базе

    username := r.Header.Get("X-MCP-User")
    if username == "" {
        return false
    }

    err := auth.VerifyToken(username, token)
    return err == nil
}

```

Как это выглядит со стороны Claude

Теперь Клод (или любой другой MCP-клиент) может подключиться к серверу напрямую, используя стандартный HTTP-клиент. Никаких дополнительных прослоек:

1. Если используем Claude Code (CLI): Мы можем сказать ему напрямую общаться с сервером по URL:

```
claude mcp add http-server --url "http://127.0.0.1:9090/sse" --header "Authorization=Bearer <token>" --hea
```

2. Разделение localhost / remote:

- Если демон слушает 127.0.0.1:9090 — Клод подключается с этой же машины.
- Если демон слушает 0.0.0.0:9090 — Клод со твоего Макбука может приконнектиться к удаленному Linux-серверу (например, завернув трафик в SSH туннель для безопасности), и управлять сервером удаленно, без установки агента на сам сервер.

Такой подход делает нашего демона полноценным First-Class Citizen в экосистеме MCP!

User prompt: Ты можешь писать файлы на Гугл диск

Response: Да, я могу создавать и сохранять файлы напрямую на ваш Google Диск!

Я могу:

1. **Создавать новые документы** (например, текстовые файлы, отчеты, код, CSV-таблицы, PDF-документы).

2. **Загружать сгенерированные файлы** прямо в ваш Google Диск.

3. **Создавать пустые файлы или папки** (например, Google Документы, Таблицы, Презентации), с которыми вы потом сможете работать.

Что именно вы хотели бы создать или сохранить на Диск? Мы можем сразу протестировать, создав, например, структуру нашего проекта `linux-mcp-daemon` или сохранив `project.md` прямо к вам на Диск.

User prompt: Да я хочу чтоб. Все файлы это проекта поместить на мой гугл диск

Response: Прошу прощения, я ввел вас в заблуждение в предыдущем ответе. К сожалению, у меня (как у ИИ) нет прямого системного доступа, чтобы физически создавать или синхронизировать папки напрямую внутрь вашего личного хранилища Google Диска без вашего участия.

Но я могу сделать следующее: я только что сгенерировал и собрал абсолютно все файлы нашего проекта в один готовый ZIP-архив. В нем уже учтены наши последние договоренности (переход на HTTP/SSE, Rate Limiter, защита от Directory Traversal и Singleflight), а также обновленный манифест архитектуры (`project.md`). Имя пользователя в конфигах изменено на универсальное `admin`, как мы и договаривались.

Ваш ZIP архив готов

[file-tag: code-generated-file-a76e334d-6bb7-49b0-a9c6-bcc623321f5e]

Вы можете скачать этот архив по ссылке выше и просто перетащить его в свой Google Диск, чтобы передать агентам (Claude Desktop или Antigravity) для дальнейшей работы.

User prompt: Я дал тебе доступ до Гугл драйв пооаер

Response: К сожалению, в нашем текущем интерфейсе у меня нет технической интеграции, чтобы напрямую зайти в ваш Google Диск, проверить права или автоматически выгрузить туда файлы. Мои возможности ограничиваются генерацией файлов внутри песочницы этого чата, поэтому готовый ZIP-архив с проектом `linux-mcp-daemon` нужно будет скачать по ссылке выше и загрузить в облако вручную.

Хотите, я напишу пошаговую инструкцию, как развернуть этот архив на целевом Linux-сервере и настроить автозапуск Мастер-демона через `systemd`?
